UNIT 3: ARRAYS

Prachi Surlakar
Assistant Professor
Computer Engineering Department
Goa College of Engineering

Unit - 3	**Array in Python:** Advantages of arrays, creating, importing, indexing and slicing, processing of array, types of array, working with single and multi-dimensional arrays using numpy, creating array using array() functions, mathematical operations on array like: addition and multiplication	**12**
	Strings and Characters: Creating, length, indexing, slicing, repeating, concatenation, comparing of strings, checking membership, removing spaces, finding substring, counting substring, changing case.	

2. Dr. R. Nageswara Rao; "Core Python Programming", Dreamtech press, Third edition, 2018.

ARRAYS IN PYTHON

- Array is an object that stores a group of elements of same datatype.
- Arrays can store only one type of data.
- Arrays can increase or decrease their size dynamically.

Advantages of arrays

- Arrays **use less memory than lists and they offer faster execution than lists**.
- **No need to specify** how many elements we are going to store into an array in the **beginning**.
- **Arrays can grow or shrink in memory dynamically**.
- **Methods** that are useful to process the elements of any array are **available in array module**

ARRAYS IN PYTHON

Creating an Array:

```
arrayname = array(type code, [elements])
```

Typecode	C Type	Minimum size in bytes
'b'	signed integer	1
'B'	unsigned integer	1
'i'	signed integer	2
'I'	unsigned integer	2
'l'	signed integer	4
'L'	unsigned integer	4
'f'	floating point	4
'd'	double precision floating point	8
'u'	unicode character	2

Examples:

a= array(' i ',[4, 6, 2, 9])

arr=array('f', [1.5 ,-2.2 ,3 6.75]

IMPORTING THE ARRAY MODULE

There are three ways to import the array module into our program.

Method 1:

```
import array
a= array.array('i',[4,6,2,9])
```

 # here first 'array' represents the module name and next array represents the class name.

method 2:

```
import array as ar
a= ar.array('i',[4,6,2,9])
```

Method 3:

```
from array import *
a=array ('i',[4,6,2,9])
```

 # * symbol represents all. The meaning is it imports all classes, objects, variables etc, from array module into our program.

IMPORTING THE ARRAY MODULE

1. Write a python program to create an integer type array and display those elements

Method 1:

```
import array
a=array.array('i',[5,6,-7,8])
print('The array elements are:')
for element in a:
    print(element)
```

Output:

```
The array elements are:
5
6
-7
8
```

Method 2:

```
from array import *
a=array('i',[5,6,-7,8])
print('The array elements are:')
for element in a:
    print(element)
```

Output:

```
The array elements are:
5
6
-7
8
```

2. Write a python program to create an array with a group of characters.

```
from array import *
a=array('u',['a','b','c','d'])
print('The array elements are:')
for element in a:
    print(element)
```

Output:

```
The array elements are:
a
b
c
d
```

3. Create an array named as 'arr1' with elements as 10,20,30,40. Create another array named 'arr2' by multiplying the elements of the first array by five. Print the elements of the 'arr2' array.

```
from array import *
arr1=array('i',[10,20,30,40])
arr2=array(arr1.typecode,(a*5 for a in arr1))
print('The array2 elements are:')
for element in arr2:
    print(element)
```

Output:

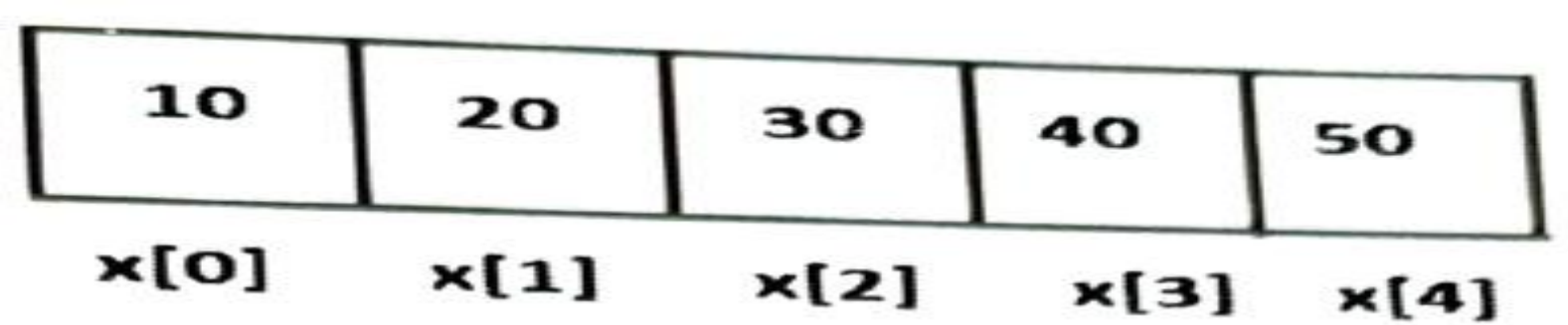
```
The array2 elements are:
50
100
150
200
```

INDEXING AND SLICING ON ARRAYS

Index: index represents the position number of an element in an array.

Example:
x=array ('i' ,[10, 20, 30, 40, 50])

Elements are stored as:

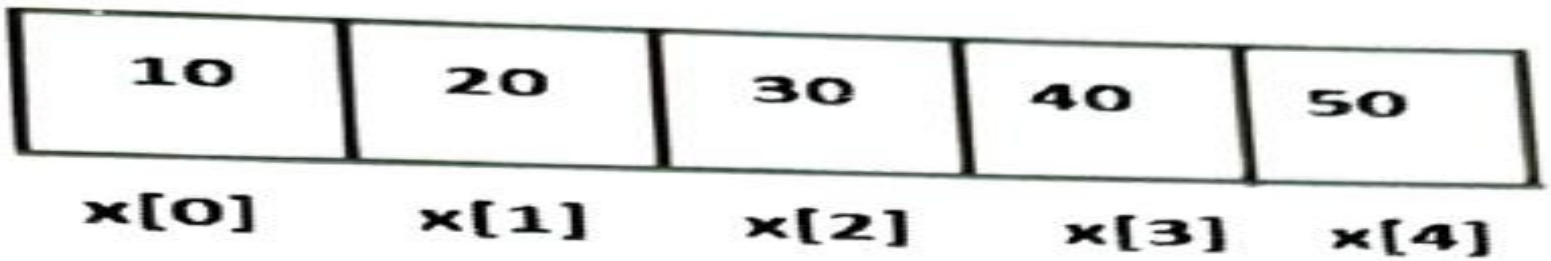


INDEXING AND SLICING ON ARRAYS

4. Write a python program to retrieve the elements of an array using array index

```
# accessing elements of an array using index
from array import *
x = array('i', [10, 20, 30, 40, 50])

# find number of elements in the array
n = len(x)

# display array elements using indexing
for i in range(n):  # repeat from 0 to n-1
    print(x[i], end=' ')
```

Explanation:



n=5

for i in range(5) #0 to 4

i	print
0	x[0]=10
1	x[1]=20
2	x[2]=30
3	x[3]=40
0	x[4]=50

INDEXING AND SLICING ON ARRAYS

Slicing operations: is used to retrieve a piece of the array that contains a group of elements.

Syntax:

arrayname(start: stop: stride)

Stride: step size excluding the starting element, optional.

SLICING ON ARRAYS

10	20	30	40	50	60	70
x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
x[-7]	x[-6]	x[-5]	x[-4]	x[-3]	x[-2]	x[-1]

```
from array import *
x=array('i',[10,20,30,40,50,60,70])
# create array y with elements from
y=x[1:4]
print(y)

#create array y with elements from 0th till last element
y=x[0:]
print(y)

#create array y with elements from 0th till 3rd
y=x[:4]
print(y)

#create array y with last four elements from x
y=x[-4:]
print(y)

# negative index starts from -1
y=x[-4:-1]
print(y)

#after 0th element, retrieve every
y=x[0:7:2]
print(y)
```

OUTPUT:

```
array('i', [20, 30, 40])
array('i', [10, 20, 30, 40, 50, 60, 70])
array('i', [10, 20, 30, 40])
array('i', [40, 50, 60, 70])
array('i', [40, 50, 60])
array('i', [10, 30, 50, 70])
```

PRACHI SURLAKAR

SLICING ON ARRAYS

10	20	30	40	50	60	70
x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
x[-7]	x[-6]	x[-5]	x[-4]	x[-3]	x[-2]	x[-1]

```
from array import *
x=array('i',[10,20,30,40,50,60,70])
# create array y with elements from
y=x[1:4]
print(y)
```

[20, 30, 40]

```
#create array y with elements from 0th till last element
y=x[0:]
print(y)
```

[10, 20, 30, 40, 50, 60, 70]

```
#create array y with elements from 0th till 3rd
y=x[:4]
print(y)
```

[10, 20, 30, 40]

```
#create array y with last four elements from x
y=x[-4:]
print(y)
```

[40, 50, 60, 70]

```
# negative index starts from -1
y=x[-4:-1]
print(y)
```

[40, 50, 60]

```
#after 0th element, retrieve every
y=x[0:7:2]
print(y)
```

[10, 30, 50, 70]

OUTPUT:

```
array('i', [20, 30, 40])
array('i', [10, 20, 30, 40, 50, 60, 70])
array('i', [10, 20, 30, 40])
array('i', [40, 50, 60, 70])
array('i', [40, 50, 60])
array('i', [10, 30, 50, 70])
```

PRACHI SURLAKAR

INDEXING AND SLICING ON ARRAYS

Slicing : can be used to create new arrays from existing arrays.
Slicing can be used to update an array.

```
from array import *
x=array('i',[10,20,30,40,50,60,70])
print(x)
x[1:3]=array('i',[5,7])
print(x)
```

OUTPUT:

```
array('i', [10, 20, 30, 40, 50, 60, 70])
array('i', [10, 5, 7, 40, 50, 60, 70])
```

INDEXING AND SLICING ON ARRAYS

Write a python program to retrieve and display only a range of elements from an array using slicing.

```
from array import *
x=array('i',[10,20,30,40,50,60,70])
for i in x[2:5]:
    print(i)
```

OUTPUT:

```
30
40
50
```

Determine the output of the following code

```
from array import *
x=array('i',[100,30,450,70,90, 60, 222])
y=x[-4:-1]
print(y)

y=x[1:4]
print(y)

y=x[0:]
print(y)

y=x[0:7:2]
print(y)

y=x[:4]
print(y)

y=x[-3:]
print(y)
```

OUTPUT:

```
array('i', [70, 90, 60])
array('i', [30, 450, 70])
array('i', [100, 30, 450, 70, 90, 60, 222])
array('i', [100, 450, 90, 222])
array('i', [100, 30, 450, 70])
array('i', [90, 60, 222])
```

Table 7.2: Array Methods

Method	Description
a.append(x)	Adds an element x at the end of the existing array a.
a.count(x)	Returns the numbers of occurrences of x in the array a.
a.extend(x)	Appends x at the end of the array a. 'x' can be another array or an iterable object.
a.fromfile(f, n)	Reads n items (in binary format) from the file object f and appends to the end of the array a. 'f' must be a file object. Raises 'EOFError' if fewer than n items are read.
a.fromlist(lst)	Appends items from the lst to the end of the array. 'lst' can be any list or iterable object.
a.fromstring(s)	Appends items from string s to the end of the array a.
a.index(x)	Returns the position number of the first occurrence of x in the array. Raises 'ValueError' if not found.
a.insert(i, x)	Inserts x in the position i in the array.
a.pop(x)	Removes the item x from the array a and returns it.
a.pop()	Remove last item from the array a.
a.remove(x)	Removes the first occurrence of x in the array a. Raises 'ValueError' if not found.
a.reverse()	Reverses the order of elements in the array a.
a.tofile(f)	Writes all elements to the file f.
a.tolist()	Converts the array 'a' into a list.
a.tostring()	Converts the array into a string.

Write a python program to understand the different methods of arrays

```
# operations on arrays
from array import*

# create an array with int values
arr = array('i', [10,20,30,40,50])
print('Original array: ', arr)

# append 30 to the array arr
arr.append(30)
arr.append(60)
print('After appending 30 and 60: ', arr)

# insert 99 at position number 1 in arr
arr.insert(1, 99)
print('After inserting 99 in 1st position: ', arr)

# remove an element from arr
arr.remove(20)
print('After removing 20: ', arr)

# remove last element using pop() method
n = arr.pop()
print('Array after using pop(): ', arr)
print('Popped element: ', n)

# finding position of element using index() method
n = arr.index(30)
print('First occurrence of element 30 is at: ', n)

# convert an array into a list using tolist() method
lst = arr.tolist()
print('List: ', lst)
print('Array: ', arr)
```

OUTPUT:

```
Original array:  array('i', [10, 20, 30, 40, 50])
After appending 30 and 60:  array('i', [10, 20, 30, 40, 50, 30, 60])
After inserting 99 in 1st position:  array('i', [10, 99, 20, 30, 40,
    50, 30, 60])
After removing 20:  array('i', [10, 99, 30, 40, 50, 30, 60])
Array after using pop():  array('i', [10, 99, 30, 40, 50, 30])
Popped element:  60
First occurrence of element 30 is at:  2
List:  [10, 99, 30, 40, 50, 30]
Array:  array('i', [10, 99, 30, 40, 50, 30])
```

Write a python program to store students marks into an array and find total marks and percentage of marks.

```
from array import *
# accept marks from keyboard into a list
lst = [int(i) for i in input('Enter marks: ').split(',')]

# create an integer array from the above list
marks = array('i', lst)

# display the marks and find total
sum=0
for x in marks:
    print(x)
    sum+=x
print('Total marks: ', sum)

# display percentage
n = len(marks)    # n = no. of elements in marks array
percent =  sum/n
print('Percentage: ', percent)
```

OUTPUT:

```
Enter marks: 50, 56, 75, 67, 45
50
56
75
67
45
Total marks: 293
Percentage: 58.6
```

Write a program to sort elements of the array using bubble sort technique.

Bubble sort example

```
Original array: 1, 5, 4, 3, 2
Compare 1 with other elements
We get: 1, 5, 4, 3, 2
Compare 5 with other elements
We get: 1, 4, 3, 2, 5
Compare 4 with other elements
We get: 1, 3, 2, 4, 5
Compare 3 with other elements
We get: 1, 2, 3, 4, 5
```

Example1:

a =10 and b= 20

Solution:

Take a temporary variable

t= a

a=b

b =t

t =a **(10)** → **t =10**

a=b**(20)** → **a=20**

b=t **(10)** → **b=10**

a=20 and b=10 (SWAPPED!!)

Write a program to sort elements of the array using bubble sort technique.

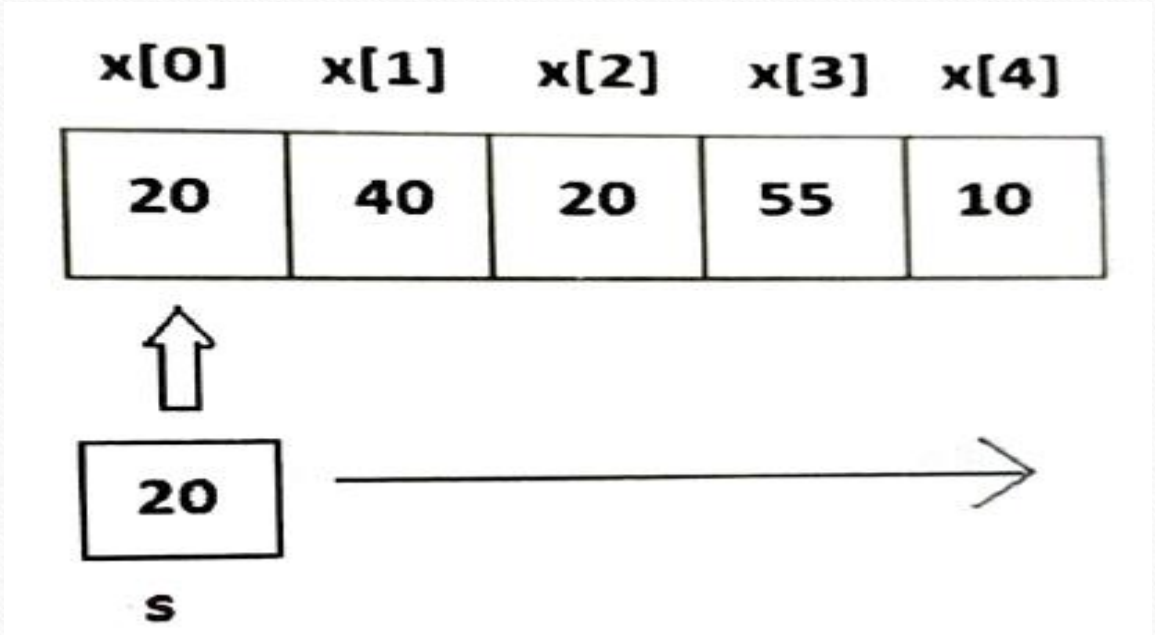
```
# sorting an array using Bubble sort technique
from array import*
# create an empty array to store integers
x = array('i', [])

# store elements into the array x
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n):  # repeat for n times
    print('Enter element: ', end='')
    x.append(int(input()))  # add the element to the array x

print('Original array: ', x)

# bubble sort
flag = False  # when swapping is done, flag becomes True
for i in range(n-1):  # i is from 0 to n-1
    for j in range(n-1-i):  # j is from 0 to one element lesser than i
        if x[j] > x[j+1]:  # if 1st element is bigger than the 2nd one
            t = x[j]  # swap j  and j+1  elements
            x[j] = x[j+1]
            x[j+1] = t
            flag = True  # swapping done, hence flag is True
    if flag==False:  # no swapping means array is in sorted order
        break  # come out of inner for loop
    else:
        flag = False  # assign initial value to flag

print('Sorted array= ', x)
```

OUTPUT:

```
Enter element: 1
Enter element: 5
Enter element: 4
Enter element: 3
Enter element: 2
Original array:  array('i', [1, 5, 4, 3, 2])
Sorted array:  array('i', [1, 2, 3, 4, 5])
```

Write a program to search for the position of an element in an array using sequential search

```
if s == x[i]:
    print('Found at Position= ', i+1)
```



```
# Searching an array for an element.
from array import*
# create an empty array to store integers
x = array('i', [])

# store elements into the array x
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n):  # repeat for n times
    print('Enter element: ', end='')
    x.append(int(input())) # add the element to the array x

print('Original array: ', x)

print('Enter element to search: ', end='')
s = int(input())  # accept element to be searched

flag=False  # this becomes True if element is found

for i in range(len(x)):  # repeat i from 0 to no. of elements
    if s == x[i]:
        print('Found at Position= ', i+1)
        flag=True
if flag==False:
    print('Not found in the array')
```

Write a program to search for the position of an element in an array using sequential search

OUTPUT:

```
How many elements? 5
Enter element: 20
Enter element: 40
Enter element: 20
Enter element: 55
Enter element: 10
Original array: array('i', [20, 40, 20, 55, 10])
Enter element to search: 20
Found at Position= 1
Found at Position= 3
```

Write a program to search for the position of an element in an array using index( ) method

```
from array import*
# create an empty array to store integers
x = array('i', [])

# store elements into the array x
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n):   # repeat for n times
    print('Enter element: ', end='')
    x.append(int(input())) # add the element to the array x

print('Original array: ', x)

print('Enter element to search: ', end='')
s = int(input())  # accept element to be searched

# index() method gives the location of the element in the array
try:
    pos = x.index(s)
    print('Found at Position= ', pos+1)

except ValueError:  # if element not found then ValueError will rise
    print('Not found in the array')
```

Write a program to search for the position of an element in an array using index( ) method

OUTPUT

```
How many elements? 5
Enter element: 20
Enter element: 40
Enter element: 20
Enter element: 55
Enter element: 10
Original array: array('i', [20, 40, 20, 55, 10])
Enter element to search: 20
Found at Position= 1
```

TYPES OF ARRAYS

SINGLE DIMENSIONAL ARRAYS:

- **These arrays represent only one row or one column of elements.**
- **Example:**
  marks= array('i', [50,60,70,66,72])

MULTIDIMENSIONAL ARRAYS:

- **These arrays represent more than one row and more than one column of elements.**

```
marks = array([[50, 60, 70, 66, 72],
               [60, 62, 71, 56, 70],
               [55, 59, 80, 68, 65]])
```

- A two dimensional array is a combination of several single dimensional arrays. A three dimensional array is a combination of several two dimensional arrays.
- **We can construct multi dimensional arrays using third party packages like numpy(numerical python).**

WORKING WITH ARRAYS USING NUMPY

- **numpy is a package that contains several classes, functions, variables etc. to deal with scientific calculations in python.**
- **Create and process single and multi dimensional arrays.**
- Contains large library of mathematical functions like linear algebra functions and Fourier transforms.
- The **arrays which are created using numpy** are called **n dimensional arrays where n can be any integer**.
- **If n=1 it represents one dimensional array. If n=2 it represents two dimensional array , if n=3 it represents three dimensional arrays.**

- Three methods to work with numpy
- **Method 1: import numpy**

Python program to create single dimensional array using numpy

```
# creating single dimensional array using numpy
import numpy
arr = numpy.array([10, 20, 30, 40, 50])  # create array
print(arr) # display array
```

OUTPUT:

```
[10, 20, 30, 40, 50]
```

WORKING WITH ARRAYS USING NUMPY

- **Method 2: import numpy as np**

```
# creating single dimensional array
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr) # display array
```

- **Method 3: from numpy import ***

```
from numpy import *
arr = array([10, 20, 30, 40, 50])
print(arr) # display array
```

Creating arrays using array( )

When we create an array we can specify the datatype of the element either as ‘int’ , ‘float’ or dtype=string.

```
arr = array([10, 20, 30, 40, 50], int)
```
integer array. We can also eliminate int

```
arr = array([1.5, 2.5, 3, 4, -5.1], float)
```
float array

```
arr = array([10, 20, 30.1, 40])
```
#If one element belongs to float type, then it will convert all other elements also to float type.

```
arr = array(['a', 'b', 'c', 'd'])
```
#create array with character type elements

arr= array([‘Delhi’ , ‘Goa’ , ‘ Mumbai’ , ‘Hyderabad’] , dtype=str) #we can store a group of strings

arr= array([‘Delhi’ , ‘Goa’ , ‘ Mumbai’ , ‘Hyderabad’]) #we can omit dtype=str

Creating arrays using array( )

Write a python program to create a string type array using numpy.

```
# creating an array with characters
from numpy import *
# create array
arr = array(['Delhi', 'Hyderabad', 'Mumbai', 'Ahmedabad'], dtype=str)
print(arr) # display array
```

Creating arrays using array( )

Write a python program to create an array from another array.

```
# creating an array from another array
from numpy import *
a = array([1,2,3,4,5])  # original array
b = array(a)  # create b from a using array() function
c = a  # create c by assigning a to c

# display the arrays
print("a = ", a)
print("b = ", b)
print("c = ", c)
```

OUTPUT:

```
a =  [1 2 3 4 5]
b =  [1 2 3 4 5]
c =  [1 2 3 4 5]
```

MATHEMATICAL OPERATIONS ON ARRAYS

It is possible to perform various mathematical operations on the elements of the array.

Example 1:

```
arr = array([10, 20, 30.5, -40])   # create an array
arr = arr+5  # add 5 to the array
```

```
# [15.  25.  35.5  -35. ]
```

Example 2:

```
arr1 = array([10, 20, 30.5, -40])
arr2 = array([1, 2, 3, 4])
arr3 = arr1 - arr2
```

```
#[ 9.  18.  27.5  -44. ]
```

These kind of operations are called as vectorized operations. Vectorized operations are important because:

- Vectorized operations are faster.
- Vectorized operations are Syntactically clearer.
- Vectorized operations provide compact code.

Table 7.4: Useful Mathematical Functions in numpy

Function	Meaning
sin(arr)	Calculates sine value of each element in the array 'arr'.
cos(arr)	Calculates cosine value of each element in the array 'arr'.
tan(arr)	Calculates tangent value of each element in the array 'arr'.
arcsin(arr)	Calculates sine inverse value of each element in the array 'arr'.
arcos(arr)	Calculates cosine inverse value of each element in the array 'arr'.
arctan(arr)	Calculates tangent inverse value of each element in the array 'arr'.
log(arr)	Calculates natural logarithmic value of each element in the array 'arr'.
abs(arr)	Calculates absolute value of each element in the array 'arr'.
sqrt(arr)	Calculates square root value of each element in the array 'arr'.
power(arr, n)	Returns power value of each element in the array 'arr' when raised to the power of 'n'.
exp(arr)	Calculates exponentiation value of each element in the array 'arr'.
sum(arr)	Returns sum of all the elements in the array 'arr'.
prod(arr)	Returns product of all the elements in the array 'arr'.
min(arr)	Returns smallest element in the array 'arr'.
max(arr)	Returns biggest element in the array 'arr'.

Function	Meaning
mean(arr)	Returns mean value (average) of all elements in the array 'arr'.
median(arr)	Returns median value of all elements in the array 'arr'.
var(arr)	Returns variance of all elements in the array 'arr'.
cov(arr)	Returns covariance of all elements in the array 'arr'.
std(arr)	Gives standard deviation of elements in the array 'arr'.
argmin(arr)	Gives index of the smallest element in the array. Counting starts from 0.
argmax(arr)	Gives index of the biggest element in the array. Counting starts from 0.
unique(arr)	Gives an array that contains unique elements of the array 'arr'.
sort(arr)	Gives an array with sorted elements of the array 'arr' in ascending order.
concatenate([a,b])	Returns an array after joining a, b arrays. The arrays a, b should be entered as elements of a list.

Python program to perform some mathematical operations on a numpy array

```
# mathematical operations on arrays
# import all from numpy module
from numpy import *

# create a numpy array using array() function
arr = array([10, 20, 30.5, -40])
print("Original array: ", arr)

# do arithmetic operations on the elements of the array
print("After adding 5: ", arr+5)
print("After subtracting 5: ", arr-5)
print("After multiplying with 5: ", arr*5)
print("After dividing with 5: ", arr/5)
print("After modulus with 5: ", arr%5)

# we can use the arrays in expressions also
print("Expression value: ", (arr+5)**2-10)
```

Python program to perform some mathematical operations on a numpy array

```
# do some math functions
print("Sin values: ", sin(arr))
print("Cos values: ", cos(arr))
print("Tan values: ", tan(arr))
print("Biggest element: ", max(arr))
print("Smallest element: ", min(arr))
print("Sum of all elements: ", sum(arr))
print("Average of all elements: ", mean(arr))
```

Python program to perform some mathematical operations on a numpy array

OUTPUT

```
Original array:  [ 10.   20.   30.5  -40. ]
After adding 5:  [ 15.   25.   35.5  -35. ]
After subtracting 5:  [  5.   15.   25.5  -45. ]
After multiplying with 5:  [  50.   100.   152.5  -200. ]
After dividing with 5:  [ 2.   4.   6.1  -8. ]
After modulus with 5:  [ 0.   0.   0.5  -0. ]
Expression value:  [  215.   615.  1250.25  1215. ]
Sin values:  [-0.54402111  0.91294525  -0.79312724  -0.74511316]
Cos values:  [-0.83907153  0.40808206  0.60905598  -0.66693806]
Tan values:  [ 0.64836083  2.23716094  -1.30222389  1.11721493]
Biggest element:  30.5
Smallest element:  -40.0
Sum of all elements:  20.5
Average of all elements:  5.125
```

THANK YOU☺

by PRACHI SURLAKAR